

CREDIT CARD FRAUD DETECTION

Machine Learning Model Creation & Deployment Project

Student: Fiza Kamal

Model: Random Forest Classifier

Task: Binary Classification

Deployment: Netlify

Repository: github.com/fizakamal003-git/credit-card-fraud-app

Report Date: 13 August 2026

Prepared to match the required 14-section professional report structure in the Machine Learning Model Creation & Deployment assignment.

FINALIZATION NOTE: Dataset source/license metadata, class/section, and any separately required Connector evidence should be verified before submission.

1. Abstract / Executive Summary

This project develops a machine learning system for detecting potentially fraudulent credit-card transactions. The task is formulated as binary classification, where **is_fraud = 0** represents a legitimate transaction and **is_fraud = 1** represents a fraudulent transaction. The project was implemented in Python/Google Colab and uses a reproducible scikit-learn pipeline containing preprocessing and a Random Forest classifier.

The workflow includes data-quality checks, duplicate and invalid-target handling, date/time feature extraction, customer-age and customer-to-merchant distance engineering, categorical one-hot encoding, numerical imputation and scaling, a stratified 80/20 train/test split, model evaluation, model serialization, and deployment of a user-facing web interface.

The Random Forest evaluation shown in the project evidence reports approximately **99.75% accuracy, 98.94% precision, 52.25% recall, 68.38% F1-score, and 0.99 ROC-AUC**. The confusion matrix contains 67,564 true negatives, 2 false positives, 170 false negatives, and 186 true positives. These results show excellent discrimination and precision, while the recall value indicates that a substantial portion of fraudulent transactions are still missed.

The application is deployed at <https://fraud-detection-ml-project.netlify.app>. The current HTML artifact explicitly labels its browser-only estimator as Demo Mode rather than the serialized Random Forest; therefore, the deployment section of this report documents this limitation and identifies the real-model backend connection as the final technical step required for a fully reproducible production-style deployment.

2. Problem Statement

Credit-card fraud detection is a binary classification problem in which transaction-level information is used to estimate whether a transaction should be classified as legitimate or fraudulent. The practical goal is to identify suspicious transactions while minimizing both false alarms and missed fraud.

Prediction target: is_fraud

Class 0: legitimate transaction

Class 1: fraudulent transaction

The model uses transaction, merchant, customer, location, timing, category, amount, and engineered features. Because fraud is a rare class in the evaluation set, precision, recall, F1-score, ROC-AUC, and the confusion matrix are more informative than accuracy alone.

This directly follows the assignment requirement to define the real-world problem, target, input features, and supervised-learning task.

3. Dataset Description

The notebook loads a CSV dataset in Google Colab and records the original row and column counts before cleaning. The target column is **is_fraud**. The modeling workflow uses transaction and contextual fields such as merchant, category, amount, city, state, latitude, longitude, city population, job, merchant latitude/longitude, and engineered time/location features.

Item	Project information
Dataset name	Credit Card Fraud Detection dataset — verify exact source title
Source / publisher	[TO BE VERIFIED FROM THE ORIGINAL DATASET SOURCE]
Source URL	[TO BE VERIFIED — required by assignment]
Access date	13 August 2026
License	[TO BE VERIFIED IF PROVIDED BY DATASET SOURCE]
Original rows	[RUN/READ ORIGINAL DATASET SHAPE TO CONFIRM]
Original columns	[RUN/READ ORIGINAL DATASET SHAPE TO CONFIRM]
Target	is_fraud (0 = legitimate, 1 = fraud)

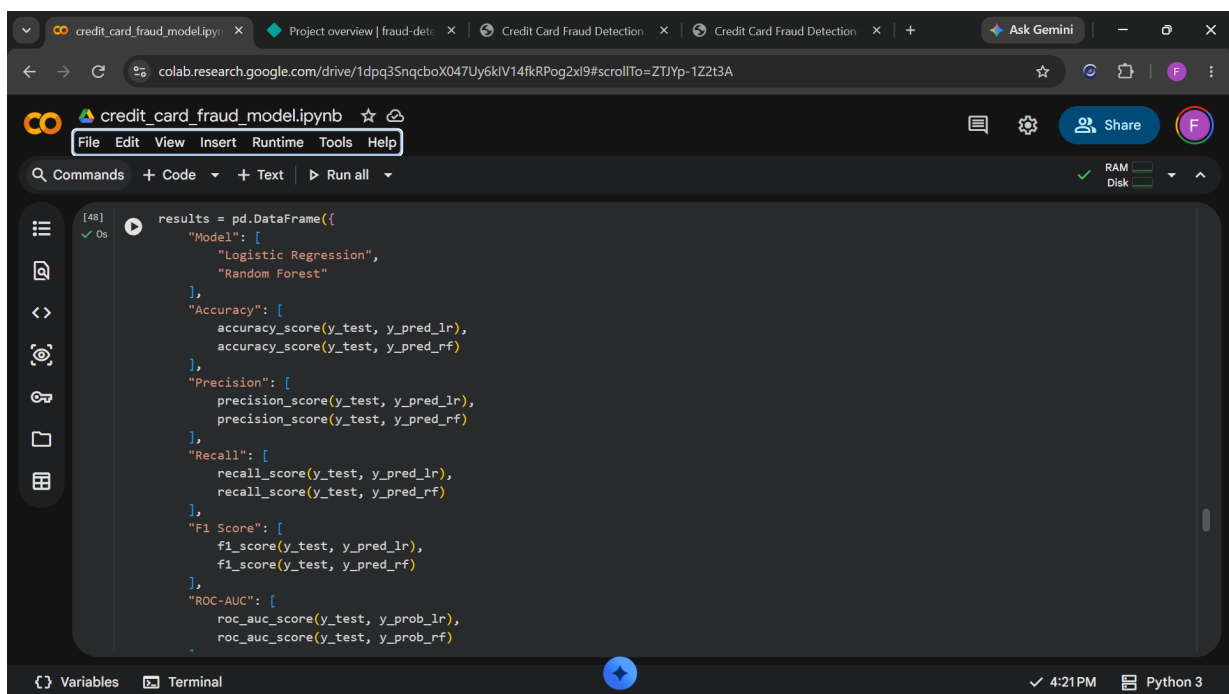
Important: The assignment specifically requires the dataset publisher/source, URL, access date, license if available, row/column counts, and feature descriptions. The exact source metadata is not present in the notebook, so those fields are intentionally left for verification rather than invented.

4. Exploratory Data Analysis

The notebook performs a structured exploratory analysis before model training. It inspects column names, data types, unique values, missing values, duplicate rows, target distribution, transaction amount statistics, fraud rates by transaction category and hour, fraud rates by state, and city-population differences.

Key EDA activities

- Dataset shape and column inspection.
- Missing-value and duplicate checks.
- Fraud/legitimate class distribution.
- Transaction amount descriptive statistics and fraud-status comparison.
- Fraud rate by category and transaction hour.
- Fraud rate by state.
- Visualization using histograms, boxplots and rate plots.

A screenshot of a Google Colab notebook interface. The browser tabs at the top include 'credit_card_fraud_model.ipynb', 'Project overview | fraud-dete...', 'Credit Card Fraud Detection', and 'Credit Card Fraud Detection'. The address bar shows a Google Drive link. The notebook title is 'credit_card_fraud_model.ipynb'. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu is a toolbar with 'Commands', '+ Code', '+ Text', and 'Run all'. On the right, there's a 'Share' button and a 'RAM Disk' indicator. The main code area shows a Python dictionary being created and assigned to 'results'. The dictionary contains keys for 'Model', 'Accuracy', 'Precision', 'Recall', 'F1 Score', and 'ROC-AUC', each with a list of values for 'lr' and 'rf' models. The code is as follows:

```
[48] results = pd.DataFrame({
    "Model": [
        "Logistic Regression",
        "Random Forest"
    ],
    "Accuracy": [
        accuracy_score(y_test, y_pred_lr),
        accuracy_score(y_test, y_pred_rf)
    ],
    "Precision": [
        precision_score(y_test, y_pred_lr),
        precision_score(y_test, y_pred_rf)
    ],
    "Recall": [
        recall_score(y_test, y_pred_lr),
        recall_score(y_test, y_pred_rf)
    ],
    "F1 Score": [
        f1_score(y_test, y_pred_lr),
        f1_score(y_test, y_pred_rf)
    ],
    "ROC-AUC": [
        roc_auc_score(y_test, y_prob_lr),
        roc_auc_score(y_test, y_prob_rf)
    ]
})
```

The bottom of the interface shows 'Variables', 'Terminal', a blue circular icon, and a status bar with '4:21 PM' and 'Python 3'.

Figure 1. Google Colab evidence of the model comparison/evaluation workflow.

5. Data Cleaning

The notebook checks for missing values and exact duplicate rows. Exact duplicate records are removed when present. Rows with a missing or invalid target are removed because they cannot be used reliably for supervised classification. The target is then converted to integer type.

Missing feature values are not manually discarded; instead, they are handled inside the reproducible preprocessing pipeline using median imputation for numerical variables and most-frequent imputation for categorical variables.

6. Categorical → Numerical Encoding

Categorical variables are transformed with OneHotEncoder using `handle_unknown='ignore'`. This converts categories such as merchant, category, city, state, and job into numerical indicator features suitable for the Random Forest pipeline. The unknown-category setting helps prevent inference failures when a new category appears at deployment time.

The encoding is fitted as part of the training pipeline rather than separately on the full dataset, helping maintain consistency between training and inference.

7. Preprocessing and Feature Engineering

The project converts the transaction timestamp to datetime and extracts transaction hour, day, month, day-of-week, and a weekend indicator. It also engineers customer age from the transaction date and date of birth, and estimates customer-to-merchant distance using a Haversine calculation.

Feature / component	Treatment
Numerical variables	Median imputation + StandardScaler
Categorical variables	Most-frequent imputation + OneHotEncoder
Timestamp	Converted to datetime; useful time components extracted
Customer age	Transaction date minus date of birth
Distance	Haversine distance between customer and merchant coordinates
Class imbalance	Random Forest uses <code>class_weight='balanced'</code>
Leakage prevention	Preprocessing is placed inside the train/test model pipeline

8. Train/Test Split

The dataset is divided into training and testing subsets using an 80/20 split. `random_state=42` is used for reproducibility, and `stratify=y` preserves the class proportions between training and testing data.

Training proportion: 80%

Testing proportion: 20%

Random state: 42

Stratification: Yes

9. Model Development

Two classification approaches are represented in the project evidence: Logistic Regression and Random Forest. Random Forest was selected as the final model because it achieved much higher precision and F1-score in the reported comparison and can capture non-linear relationships and feature interactions.

Random Forest parameter	Value
n_estimators	150
max_depth	20
min_samples_leaf	2
class_weight	balanced
random_state	42
n_jobs	-1

The complete preprocessing and classifier are stored together as a scikit-learn Pipeline. The notebook saves the final pipeline with joblib as `credit_card_fraud_random_forest.pkl` and saves the expected modeling feature names as `credit_card_fraud_features.json`.

10. Evaluation and Interpretation

Metric	Random Forest result
Accuracy	99.75%
Precision	98.94%
Recall	52.25%
F1-score	68.38%
ROC-AUC	≈ 0.99
Average Precision	Not included in the available visual evidence

Interpretation: Accuracy and precision are very high, meaning the model rarely flags a legitimate transaction as fraud and is highly precise when it predicts fraud. However, recall is only 52.25%, meaning the model detects roughly half of the fraudulent transactions in the reported test results. The F1-score reflects this trade-off. The ROC-AUC of about 0.99 indicates excellent ranking/discrimination ability across thresholds.

For fraud detection, recall is operationally important because missed fraud can have financial consequences. A future version should investigate threshold tuning, calibration, resampling or cost-sensitive optimization to improve recall while controlling false positives.

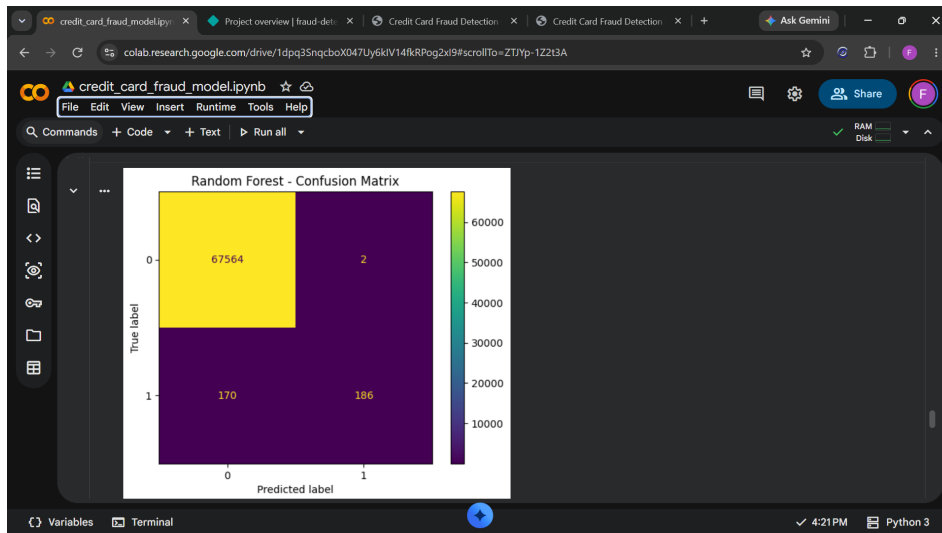


Figure 2. Random Forest confusion matrix.

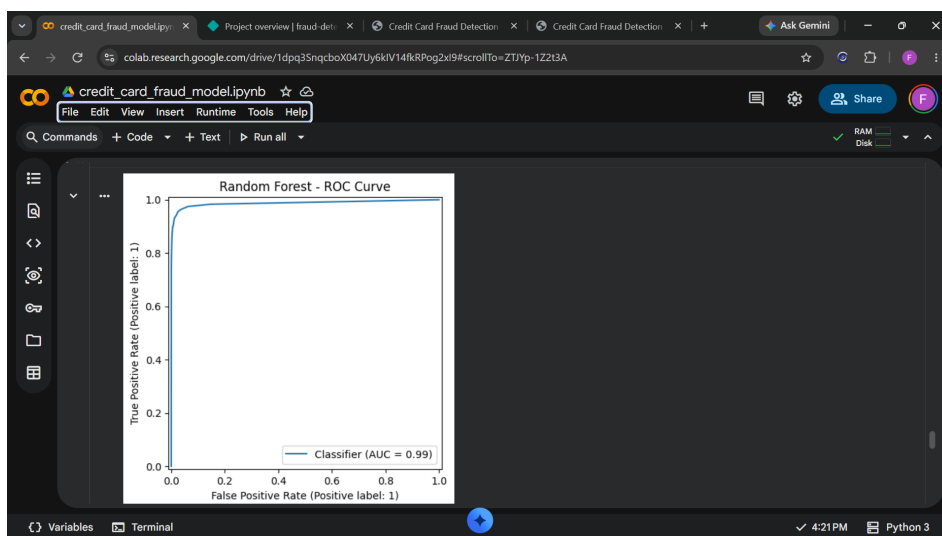


Figure 3. Random Forest ROC curve.

11. Claude Artifact / Connector / skills.md

A browser-based transaction-risk dashboard was created as an AI-assisted development artifact. The interface contains a transaction input form, a prediction/result panel, model information, risk information, and an explanation of how a production backend would connect to the saved Random Forest pipeline.

The artifact is useful for demonstrating the intended user experience and deployment workflow. However, the current HTML explicitly states that the browser artifact runs in Demo Mode because a scikit-learn .pkl file cannot be directly executed in the sandboxed browser. Its predictions therefore come from a transparent rule-based estimator rather than the trained Random Forest.

Connector evidence: No separate Connector workflow is documented in the available project files. If the course requires a Connector demonstration, add a screenshot and short explanation showing the service/tool used, what information flowed through it, and why it was useful.

skills.md: The assignment requires a skills.md file describing reusable AI-assisted project workflow instructions. This file should be added to the GitHub repository before final submission.

12. Deployment on Netlify

The user-facing application is deployed on Netlify and was tested in a browser. The interface provides the project title, a short description, transaction input fields, a prediction/result area, and model information, matching the minimum interface requirements in the assignment.

Deployment item	Value
Platform	Netlify
Live URL	https://fraud-detection-ml-project.netlify.app
Repository	https://github.com/fizakamal003-git/credit-card-fraud-app
Frontend	HTML/CSS/JavaScript dashboard
Model artifact	fraud_detection_random_forest.pkl
Current browser inference	Demo Mode rule-based estimator

Deployment limitation: The current HTML source contains a `runRealModel()` placeholder and falls back to `demoModeEstimate()`. Therefore, the public page is a functioning UI, but it does not currently send the user's input to the serialized Random Forest model. For the final assignment submission, the recommended architecture is a small backend API that loads the .pkl with `joblib` and exposes a `POST /predict` endpoint, with the Netlify frontend calling that API.

13. Conclusion & Future Work

This project demonstrates an end-to-end machine learning workflow for credit-card fraud classification: problem definition, data quality checks, feature engineering, preprocessing, stratified train/test splitting, Random Forest training, evaluation, model serialization, and web deployment.

The final Random Forest shows strong overall discrimination and precision, but its recall indicates that further work is needed before the model should be treated as a high-recall fraud screening system. The project also demonstrates an important engineering lesson: a trained model file and a web interface are not automatically connected. The inference path must explicitly load the same trained pipeline and return predictions from that model.

Recommended future improvements

- Connect the Netlify frontend to a real prediction API that loads the saved Random Forest pipeline.
- Tune the classification threshold to improve fraud recall according to the desired business cost trade-off.
- Compare additional models and document cross-validation/hyperparameter selection.
- Investigate class imbalance using additional cost-sensitive or resampling strategies.

- Add model monitoring, input validation, logging, and calibrated probabilities for a production setting.

14. References

1. Machine Learning Model Creation & Deployment Assignment, course assignment document supplied for this project.
2. Project notebook: `Copy_of_Untitled12_Random_Forest_Corrected.ipynb`.
3. scikit-learn documentation for RandomForestClassifier, Pipeline, ColumnTransformer, OneHotEncoder, StandardScaler, and classification metrics.
4. Netlify deployment: <https://fraud-detection-ml-project.netlify.app>
5. GitHub repository: <https://github.com/fizakamal003-git/credit-card-fraud-app>
6. Dataset source: [ADD THE ORIGINAL DATASET PUBLISHER, URL, VERSION/DATE, AND LICENSE BEFORE SUBMISSION].

Final Submission Checklist

The assignment requires a complete GitHub repository, live Netlify URL, PDF report, skills.md, Claude Artifact/Connector evidence, and reproducible project files.

✓	Problem definition and target documented
✓	EDA and data-quality checks documented
✓	Data cleaning and feature engineering documented
✓	Categorical encoding and preprocessing documented
✓	80/20 stratified train/test split documented
✓	Random Forest trained and saved
✓	Evaluation metrics and confusion matrix documented
✓	Netlify URL included
✓	GitHub repository URL included
■	Verify dataset publisher/source URL/license/row-count metadata
■	Add skills.md to GitHub repository
■	Add Connector evidence if required by your class
■	Connect the Netlify frontend to the real Random Forest API before claiming live ML inference

This report deliberately avoids inventing dataset metadata or deployment claims that are not supported by the project files. Replace the marked verification items before final submission.